

Public Key Cryptography

Public Key Cryptography

Kerckhoff's Principle: the cryptosystem (algorithm) is public

Symmetric or shared key: only the key is secret, used both for encryption and decryption. Shared keys for MACs, PRFs, etc..

Asymmetric cryptographic capability: encryption capability without decryption capability.

Public Key Cryptosystem (PKC)

Kerckhoff: cryptosystem (algorithm) is public

DH76: can encryption key be public too?

- Decryption key will be different (and private)
- Everybody can send me emails, but only I can read them.

Is it Only About Encryption?

Also: digital signatures for integrity and non-repudiation.

Sign with private key s , verify with public key v .

Recall MACs: a shared key cryptosystem for message authentication

Private signing key, public validation key

More: Key-Exchange Protocols

Establish shared key between Alice and Bob without assuming an existing shared (master) key

Use public information from Alice and Bob to setup shared key k

Eavesdroppers cannot learn the key k

Public keys solve more problems

Signatures provide evidence (everyone can validate, only owner can sign)

Established shared secret keys

- Use authenticated public keys, signed by trusted certificate authority
- Or use DH (Diffie Hellman) key exchange

Stronger resiliency to key exposure

- Perfect forward secrecy and recover security
- Threshold security: resilient to key exposure of t out of n parties

Public keys are easier... To distribute: from directory or from incoming message (still require authentication), and less keys to distribute

To maintain: can keep in non-secure storage as long as being validated before using. Also less keys: $O(|parties|)$ versus $O(|parties|^2)$

So why not always use public key crypto?

The Price of PKC

Assumptions: Applied PKC algorithms are based on a small number of specific computational assumptions

- Mainly: hardness of factoring and discrete-log
- Both may fail against quantum computers

Overhead:

- Computational
- Key length (public key is long)
- Output length (e.g. ciphertext or signature)

Public key crypto is harder...

Requires related public, private keys

- Usually we say a keypair (pk, sk)
- Public key does not expose private key

Substantial overhead

- Successful cryptanalytic shortcuts → need long keys
- Elliptic curves (EC) may allow shorter keys (almost no shortcuts found)
- Complex computations, e.g. complex (slow) keygen

In summary

Minimize use of PKC

If possible, minimize length of the input

How?

For signatures, hash-then-sign

For public-key encryption, hybrid encryption

Hybrid Encryption

Challenge: public key cryptosystems are slow

Hybrid encryption:

- Use a shared key encryption scheme to encrypt all messages,
- But use a public key encryption scheme to exchange the shared key
 - Alice generates k , encrypts it under Bob's public key and sends the ciphertext c_k to Bob.
 - Bob can decrypt and recover k , and use k to decrypt c_M

Number Theory Review

$$(a + b) \bmod m = [(a \bmod m) + (b \bmod m)] \bmod m$$

Multiplicative Inverse

To compute $a/c \bmod m$, multiply a by the multiplicative inverse of c

That is, compute $a/c \bmod m = ac^{-1} \bmod m$

Where c^{-1} is the multiplicative inverse such that $cc^{-1} \bmod m = 1$

Not all ints have multiplicative inverses with respect to a specific modulus m .

Modular Exponentiation

Encountered a lot: discrete log scheme, RSA, etc

Fermat's Little Theorem (for prime modulus):

$$a^b \bmod p = (a \bmod p)^{b \bmod (p-1)} \bmod p$$

Group Theory Review

A group G is a cyclic if there is an element $g \in G$ s.t. for every element $a \in G$, there is an int i s.t. $a = g^i$. Such an element g is called a generator of G . The order of G is the total.

Key Exchange

Defining a Key Exchange Protocol

Must satisfy:

Correctness: both parties compute the same shared key

Key Indistinguishability: the key that the two parties establish is indistinguishable from random

Discrete Log Problem

A computationally hard problem that is hard to solve but easy to verify

Given a generator g and an element $a \in G$, find i s.t. $a = g^i$. Verification: exponentiation (efficient algorithm)

Computing log is quite efficient over the real numbers. But is discrete-log hard?

- Weak, where it's not hard: prime p where $(p-1)$ has only 'small' prime factors, mistakes/trapdoors found
- Safe-prime groups: $p = 2q + 1$, where q is sufficiently large

Group needs to be big, cyclic with generator, and discrete log problem within group to be hard.

Game: pick random element in group. First, pick the power (random int) and then compute the element as the generator to that power. Attacker is given g^y , where they must compute y .

y is the secret key, the public key is g^y .

The Diffie-Hellman (DH) Key Exchange Protocol and The Computational/Decisional Diffie-Hellman Assumptions (CDH/DDH)

DH Key-Exchange

Setup: agree on a random safe (large) prime p and generator g for the cyclic multiplicative group Z_p^* .

Alice: pick at random a secret int a from Z_p^* , then compute $P_A = g^a \mod p$, and send P_A to Bob.

Bob: pick at random secret int b from Z_p^* , then compute $P_B = g^b \mod p$, and send P_B to Alice.

Both parties: compute the shared key $k = g^{ab} \mod p$

$(g^b)^a = g^{ab} = (g^a)^b \mod p$; Alice raises Bob's g^b to the a^{th} power, and vice versa.

Correctness: two parties establish identical, long-enough, and random key.

Caution: Auth the Public Keys!

DH Key-Exchange is only secure against eavesdroppers, but not MitM attackers (since they can change the messages)

So, the public messages being sent must be authenticated, e.g. using digital signatures.

Security of DH Key Exchange

Assume authenticated comms.

DH key exchange requires stronger assumption than discrete log

The Computational Diffie-Hellman (CDH) Assumption: given $g^b \mod p$ and $g^a \mod p$, an efficient adversary cannot compute $g^{ab} \mod p$ with non-negligible probability.

So DH key exchange protocol is secure for groups in which the CDH assumptions holds.

Using DH Securely?

Can g^a , g^b expose *something* about $g^{ab} \mod p$? Finding at least one bit about it is easy

Option 1: use DH but with a stronger group where g^a, g^b, g^c are indistinguishable

Option 2: use DH with Z and safe prime (where only CDH holds), but use a key derivation function to derive a secure shared key (like a randomness-extracting hash function).